X-CAVATE

Concentric Tube Robot Needle Simulation, Auto-Placement Optimization, & Gimbal Integration

Comprehensive Engineering Change Report

Branch: `claude/serene-proskuriakova`

Date:	March 12, 2026
Author:	Soham S.
Platform:	macOS Darwin 25.3.0
Python:	3.x (Conda environment)
Visualization:	Plotly + Streamlit

This report documents all changes implemented across the X-CAVATE codebase to support CTR needle simulation, automated base placement optimization, gimbal workspace expansion, and Streamlit GUI integration.

Contents

1 Executive Summary **2**

2 Hardware Context **2**

3 New File: CTR Needle Simulation Engine **3**
3.1 Overview . . . 3
3.2 Forward Kinematics: `compute_needle_geometry()` . . . 3
3.3 Gimbal Frame: `compute_gimbal_frame()` . . . 4
3.4 Single-Pass Animation: `create_ctr_simulation()` . . . 4
3.5 All-Passes Animation: `create_ctr_simulation_all_passes()` . . . 5
3.6 Frame Data Helper: `_compute_frame_data()` . . . 6
3.7 Non-Gimbal Fallback: `build_fallback_solutions()` . . . 6

4 New File: CTR Auto-Placement Optimizer **6**
4.1 Overview . . . 6
4.2 Function: `optimize_ctr_placement()` . . . 7

5 Modified File: Pipeline Orchestrator **8**
5.1 Overview . . . 8
5.2 Change 1: Safe CTR Config Scoping (Line 156) . . . 8
5.3 Change 2: Auto-Placement Integration (Step 3b) . . . 8
5.4 Change 3: Empty Pass Guard in `_compute_instructions()` . . . 8
5.5 Change 4: Simulation Data in Return Dict . . . 9

6 Modified File: Streamlit GUI Application **9**
6.1 Overview . . . 9
6.2 Change 1: Gimbal Toggle (Top-Level) . . . 9
6.3 Change 2: CTR Default Configuration . . . 10
6.4 Change 3: Auto-Optimize CTR Placement . . . 10
6.5 Change 4: CTR Needle Simulation Section . . . 10
6.6 Change 5: Config Building . . . 11

7 Modified File: Configuration Dataclass **11**
7.1 Overview . . . 11
7.2 Default Value Updates . . . 12

8 Modified File: Plotting Module **12**
8.1 Overview . . . 12
8.2 Empty Pass Guard in `create_network_plot_merged()` . . . 12

9 Bugs Discovered and Fixed **12**
9.1 Coordinate Scaling Mismatch (Critical) . . . 12
9.2 KeyError on Pass Index (Medium) . . . 13
9.3 Empty Sequence on Min (Medium) . . . 13
9.4 Empty Array Concatenation (Low) . . . 13

10 Performance Considerations **13**
10.1 HTML File Size Optimization . . . 13
10.2 Trail Subsampling . . . 13
10.3 Auto-Placement Optimizer . . . 13

11 Gimbal Workspace Analysis **14**
11.1 Problem: Fringe Node Pruning . . . 14
11.2 Solution: Gimbal Cone Tilt . . . 14
11.3 Result . . . 14

12 Data Flow Architecture **14**

13 Key Data Structures **15**
13.1 `GimbalNodeSolution` . . . 15
13.2 `ctr_config_params` . . . 15

14 Testing and Validation **15**
14.1 Pipeline Test: `finale.txt` . . . 15
14.2 Without Gimbal (Baseline) . . . 16
14.3 Simulation Verification . . . 16

15 Complete File Inventory **16**

16 Conclusion **16**

1 Executive Summary

This report documents a comprehensive set of engineering changes to the X-CAVATE vascular network printing platform. The changes introduce three major capabilities:

1. **Animated CTR Needle Simulation** — Real-time 3D visualization of the concentric tube robot mechanism (stainless steel outer tube, pre-curved nitinol inner tube, gimbal tilt) as it prints node-by-node through the vascular network. Supports both single-pass and all-passes animation modes.
2. **Automated CTR Base Placement Optimization** — Spherical shell sampling algorithm that automatically determines the optimal CTR base position and insertion direction to maximize the number of reachable network nodes.
3. **Gimbal Workspace Expansion** — Integration of the gimbal solver into the simulation pipeline and GUI, enabling the CTR to tilt its insertion direction within a configurable cone (default 15° half-angle) to reach fringe nodes that would otherwise fall outside the workspace boundary.

These features span **6 files** (2 new, 4 modified), adding approximately **1,200 lines** of new production code and modifying approximately **200 lines** of existing code.

Table 1: Files Changed Summary

File	Status	Lines	Purpose
`xcavate/viz/ctr_simulation.py`	r	861	Needle animation engine
`xcavate/core/ctr_placement.py`	r	174	Auto-placement optimizer
`xcavate/pipeline.py`	**[MODIFIED]**	+45	Expose simulation data
`xcavate/gui/app.py`	**[MODIFIED]**	+120	GUI integration
`xcavate/config.py`	**[MODIFIED]**	+5	Hardware defaults update
`xcavate/viz/plotting.py`	**[MODIFIED]**	+8	Empty-pass guard

2 Hardware Context

The changes are designed around a specific concentric tube robot (CTR) hardware configuration:

Table 2: CTR Hardware Specifications

Parameter	Value	Unit
Stainless steel (SS) outer tube OD	4.0	mm
Nitinol inner tube OD	1.6	mm
Pre-curved bend radius	55.0	mm
Max SS extension	65.0	mm
Max nitinol extension	140.0	mm
Gimbal cone half-angle	15.0	degrees
Gimbal tilt levels	3	—
Gimbal azimuth samples	8	—

The CTR operates by translating and rotating two concentric tubes relative to each other:

- The **stainless steel outer tube** extends straight along the insertion direction $\hat{R}$, providing rigid axial reach.

- The **nitinol inner tube** is pre-curved with radius $r = 55\,\text{mm}$. When extended beyond the SS tube tip, it deflects into a circular arc, allowing the needle tip to reach points off the insertion axis.
- **Rotation** (θ) about the insertion axis sweeps the bend plane through 360°.
- The **gimbal** tilts the entire insertion direction $\hat{R}$ by angle α at azimuth ϕ, expanding the reachable workspace.

The actuator space is $(\texttt{z_ss}, \texttt{z_ntnl}, \theta)$ for tube extensions and rotation, plus (α, ϕ) for gimbal tilt.

3 New File: CTR Needle Simulation Engine

3.1 Overview

- **File:** `xcavate/viz/ctr_simulation.py`
- **Status:** r
- **Lines:** 861
- **Functions:** 7 (3 public, 1 internal helper, 2 animation builders, 1 fallback)
- **Dependencies:** `numpy`, `plotly.graph_objects`, `GimbalNodeSolution`

This module is the core visualization engine. It computes needle geometry from actuator values using forward kinematics and builds interactive Plotly animations that show the CTR mechanism moving node-by-node through the print path.

3.2 Forward Kinematics: `compute_needle_geometry()`

Purpose: Compute the 3D centerline geometry of the full CTR needle (SS straight section + nitinol arc) given actuator values.

Signature:

```
def compute_needle_geometry(
    X: np.ndarray,              # (3,) base position
    R_hat: np.ndarray,          # (3,) insertion direction
    n_hat: np.ndarray,          # (3,) bend plane normal
    theta_match: float,         # alignment angle
    z_ss: float,                # SS extension (mm)
    z_ntnl: float,              # nitinol extension (mm)
    theta: float,               # rotation angle (rad)
    radius: float,              # bend radius (mm)
    n_ss_samples: int = 12,     # SS discretization
    n_arc_samples: int = 20,    # arc discretization
) -> dict:
```

Algorithm:

1. **SS straight section:** Generate `n_ss_samples` points along the line $\mathbf{X} + t \cdot \hat{R}$ for $t \in [0, z_{\text{ss}}]$. This produces the orange line in the visualization.
2. **Nitinol arc section:** If $z_{\text{ntnl}} > z_{\text{ss}}$ (nitinol extends beyond SS tube):
   - Compute the bend direction via simplified Rodrigues rotation:

$$\hat{K}_n = \hat{R} \times \hat{n}, \qquad \hat{d} = \hat{n}\cos(\theta_{\text{match}} + \theta) + \hat{K}_n \sin(\theta_{\text{match}} + \theta)$$

   - Compute maximum arc angle: $\alpha_{\max} = (z_{\text{ntnl}} - z_{\text{ss}})/r$
   - Generate `n_arc_samples` points along the circular arc:

$$\mathbf{p}(\alpha) = \mathbf{X}_{\text{ss_tip}} + r\sin(\alpha)\hat{R} + r(1 - \cos\alpha)\hat{d}$$

Returns: Dictionary with keys `base`, `ss_tip`, `ss_line` (array of shape $(n_{\text{ss}}, 3)$), `arc_line` (array of shape $(n_{\text{arc}}, 3)$), and `tip` (final endpoint).

3.3 Gimbal Frame: `compute_gimbal_frame()`

Purpose: Compute the tilted insertion direction and reference frame when the gimbal is actuated, and generate visualization lines for both the tilted and untilted directions.

Signature:

```
def compute_gimbal_frame(
    X: np.ndarray,              # (3,) base position
    R_hat_base: np.ndarray,     # (3,) untilted insertion direction
    n_hat_base: np.ndarray,     # (3,) untilted bend normal
    alpha: float,               # tilt angle (rad)
    phi: float,                 # azimuth angle (rad)
    frame_length: float = 15.0,  # visualization line length (mm)
) -> dict:
```

Algorithm:

1. If $\alpha = 0$: no tilt, $\hat{R}_{\text{tilted}} = \hat{R}_{\text{base}}$.
2. Otherwise, apply **Rodrigues rotation**:
   - Compute binormal: $\hat{b} = \hat{R}_{\text{base}} \times \hat{n}_{\text{base}}$, normalized.
   - Compute tilt axis: $\hat{k} = \cos(\phi)\hat{n}_{\text{base}} + \sin(\phi)\hat{b}$, normalized.
   - Construct skew-symmetric matrix $\mathbf{K}$ from $\hat{k}$.
   - Rotation matrix: $\mathbf{R}_{\text{tilt}} = \mathbf{I} + \sin(\alpha)\mathbf{K} + (1 - \cos\alpha)\mathbf{K}^2$
   - Apply: $\hat{R}_{\text{tilted}} = \mathbf{R}_{\text{tilt}}\,\hat{R}_{\text{base}}$, normalized.
3. Compute new perpendicular normal $\hat{n}_{\text{tilted}}$ and alignment angle $\theta_{\text{match,tilted}}$.
4. Generate two direction lines from the base point:
   - **Tilted direction** (cyan, solid): $[\mathbf{X}, \mathbf{X} + 15\hat{R}_{\text{tilted}}]$
   - **Base direction** (gray, dotted): $[\mathbf{X}, \mathbf{X} + 15\hat{R}_{\text{base}}]$

Returns: Dictionary with `R_hat_tilted`, `n_hat_tilted`, `theta_match_tilted`, `direction_line` (tilted), and `base_direction_line` (untilted reference).

Why 15 mm? The frame length was increased from an initial 2 mm to 15 mm so that the gimbal direction line extends visibly beyond the CTR body in the 3D scene. This makes gimbal orientation changes clearly discernible during animation playback. The untilted reference line provides a fixed baseline for visual comparison.

3.4 Single-Pass Animation: `create_ctr_simulation()`

Purpose: Build an interactive Plotly animation showing the CTR needle printing one pass node-by-node.

Signature:

```
def create_ctr_simulation(
    print_pass: List[int],
    points: np.ndarray,
    gimbal_solutions: Dict[int, GimbalNodeSolution],
    ctr_base: np.ndarray,
    R_hat_base: np.ndarray,
    radius: float,
    n_hat_base: np.ndarray,
    theta_match: float,
```

```
    max_frames: int = 200,
    trail_max_points: int = 20000,
) -> go.Figure:
```

Trace Architecture (8 traces):

Table 3: Plotly Trace Layout

#	Name	Type	Color	Static?	Description
0	Network Ghost	markers	gray, 30%	Yes	All network points (max 10K)
1	CTR Base	marker	orange diamond	Yes	Base position **X**
2	SS Tube	line	orange, w=6	No	Straight SS section
3	Nitinol Arc	line	dodgerblue, w=6	No	Curved nitinol section
4	Tip Marker	marker	lime, s=6	No	Needle tip position
5	Printed Trail	lines+markers	red, w=3	No	Accumulated print path
6	Gimbal Dir.	line	cyan, w=4	No	Tilted insertion direction
7	Base Dir.	line	gray dotted, w=2	No	Untilted reference direction

Frame Generation Algorithm:

1. **Subsample:** If the pass has more nodes than `max_frames` (default 200), compute evenly-spaced indices preserving first and last. With 18K nodes and 200 frames, step $\approx$ 90 nodes.
2. **For each frame step k:**
   (a) Look up `gimbal_solutions[node]` $\rightarrow$ get $(\alpha, \phi, z_{\text{ss}}, z_{\text{ntnl}}, \theta, \texttt{config_idx})$.
   (b) Call `compute_gimbal_frame()` to get tilted direction and reference line.
   (c) Call `compute_needle_geometry()` with tilted config to get SS line + arc + tip.
   (d) Accumulate printed trail: all pass nodes from start to current step, subsampled to `trail_max_points` if needed.
   (e) Build `go.Frame` with updated data for traces 2–7 only.
3. **Performance optimization:** Each frame uses `traces=[2,3,4,5,6,7]` to update only the 6 dynamic traces. The static ghost points (trace 0, up to 10K points) and base marker (trace 1) are never repeated in frames. This reduced the HTML file size by approximately 65% compared to the naive approach of repeating all 8 traces in every frame.

Controls:

- Play/Pause buttons (bottom-left)
- Frame slider with node index labels
- Title shows current node index, total count, and gimbal config ($\alpha°$, $\phi°$)
- Fixed axis ranges computed from network extents ± 5 mm padding
- `aspectmode="data"` for correct proportions

3.5 All-Passes Animation: `create_ctr_simulation_all_passes()`

Purpose: Build an animation covering all print passes in sequence, with per-pass trail coloring.

Key differences from single-pass:

- Concatenates all passes into a single flat timeline: `[(node, pass_idx), ...]`
- Default `max_frames` = 300, `trail_max_points` = 20,000
- Trail segments are separated by NaN rows (visual breaks between passes)
- Each pass is assigned a distinct color from an 18-color categorical palette:

```
_cmap = [
    "#e6194b", "#3cb44b", "#4363d8", "#f58231", "#911eb4",
    "#42d4f4", "#f032e6", "#bfef45", "#fabed4", "#469990",
    "#dcbeff", "#9A6324", "#800000", "#aaffc3", "#808000",
    "#ffd8b1", "#000075", "#a9a9a9",
]
```

- Frame title includes: `Pass {current}/{total} | Node {k}/{n_total} Config ...`

3.6 Frame Data Helper: `_compute_frame_data()`

Purpose: Compute needle geometry and gimbal frame for a single node, with fallback handling.

```
def _compute_frame_data(
    node, gimbal_solutions, ctr_base, R_hat_base,
    n_hat_base, theta_match_base, radius
) -> tuple:  # (geom_dict, gimbal_dict)
```

Logic:

- If node is in `gimbal_solutions`: compute tilted frame via `compute_gimbal_frame()`, then needle geometry in tilted config.
- If node is *not* in solutions (fallback): return retracted position ($z_{\text{ss}} = 0$, $z_{\text{ntnl}} = 0$, $\theta = 0$) with zero gimbal angles. The needle appears fully retracted into the base.

3.7 Non-Gimbal Fallback: `build_fallback_solutions()`

Purpose: Generate synthetic `GimbalNodeSolution` objects using analytical inverse kinematics when the gimbal solver was not enabled.

Algorithm:

1. Reconstruct the local coordinate frame from stored `ctr_config_params`: base position $\mathbf{X}$, insertion direction $\hat{R}$, bend normal $\hat{n}$, and alignment angle θ_{match}.
2. Apply Rodrigues rotation to build direction matrix $\mathbf{D} = [\hat{R}, \hat{n}_{\text{rot}}, \hat{t}]$.
3. For each unique node across all passes:
   (a) Transform target point to local coordinates: $\boldsymbol{\delta} = \mathbf{D}(\mathbf{p} - \mathbf{X})$
   (b) Compute rotation angle: $\theta = \arctan 2(\delta_z, \delta_y)$
   (c) Compute perpendicular distance: $r_\perp = \sqrt{\delta_y^2 + \delta_z^2}$
   (d) Solve arc geometry:
$$\cos\alpha = 1 - r_\perp/r, \qquad \alpha_{\text{arc}} = \arccos(\cos\alpha_{\text{clamped}})$$
$$z_{\text{ntnl}} - z_{\text{ss}} = r \cdot \alpha_{\text{arc}}, \qquad z_{\text{ss}} = \delta_x - r\sin(\alpha_{\text{arc}})$$
4. Store as `GimbalNodeSolution` with `config_idx` $= 0$, $\alpha = 0$, $\phi = 0$ (no gimbal tilt).

4 New File: CTR Auto-Placement Optimizer

4.1 Overview

- **File:** `xcavate/core/ctr_placement.py`
- **Status:** r
- **Lines:** 174
- **Functions:** 1 public
- **Dependencies:** `numpy`, `logging`, `ctr_kinematics.CTRConfig`, `ctr_kinematics.batch_is_reachable`

4.2 Function: `optimize_ctr_placement()`

Signature:

```python
def optimize_ctr_placement(
    points: np.ndarray,        # (N, 3) network node positions
    radius: float = 55.0,     # bend radius (mm)
    ss_lim: float = 65.0,     # max SS extension (mm)
    ntnl_lim: float = 140.0, # max nitinol extension (mm)
    calibration_file = None,  # optional .npy calibration
    n_azimuth: int = 12,      # azimuth samples on sphere
    n_elevation: int = 7,     # elevation samples on sphere
    n_distances: int = 4,     # radial distance samples
    standoff_min: float = 1.2,
    standoff_max: float = 2.5,
) -> dict:
```

Algorithm:

1. **Compute network geometry:**
   - Centroid: $\mathbf{c} = \text{mean}(\text{points})$
   - Bounding sphere radius: $r_{\text{bound}} = \max \|\mathbf{p}_i - \mathbf{c}\|$
   - Axial reach: $d_{\text{reach}} = z_{\text{ss,max}} + r$
2. **Generate candidate positions on spherical shell:**
   - Standoff distances: $d \in \text{linspace}(s_{\min} \cdot 0.5 \cdot r_{\text{bound}}, s_{\max} \cdot r_{\text{bound}}, n_d)$
   - Direction vectors: azimuth $\phi \in [0, 2\pi)$ (n_{az} samples), elevation $\theta \in [-70°, +70°]$ (n_{el} samples)
   - Total candidates: $n_{\text{az}} \times n_{\text{el}} \times n_d = 12 \times 7 \times 4 = 336$
3. **For each candidate position $\mathbf{X}_{\text{cand}}$:**
   (a) Compute insertion direction: $\hat{R} = (\mathbf{c} - \mathbf{X}_{\text{cand}})/\|\mathbf{c} - \mathbf{X}_{\text{cand}}\|$ (points toward centroid).
   (b) Build a `CTRConfig` with these parameters.
   (c) Evaluate `batch_is_reachable()` on all network points — returns boolean array.
   (d) Count reachable nodes. Track the best candidate.
4. **Return:** Dictionary with `position`, `orientation`, `reachable_count`, `reachable_fraction`, `total_nodes`, `candidates_tested`.

Why spherical shell sampling? The CTR has a limited axial reach ($z_{\text{ss,max}} + r \approx 120\,\text{mm}$). The base must be positioned far enough from the network for the workspace to enclose it, but not so far that nodes exceed the maximum extension. The standoff range $[1.2, 2.5] \times r_{\text{bound}}$ empirically covers the viable placement zone. Elevation is limited to $\pm 70°$ to avoid degenerate near-vertical placements.

Performance: With 336 candidates and batch reachability checks (vectorized NumPy operations), the optimizer completes in < 2 seconds for networks with 18K+ nodes.

Critical integration note: The optimizer must run on *interpolated* (post-preprocessing) coordinates, not raw input coordinates. The coordinate pipeline applies `convert_factor` (10.0, cm→mm) × `scale_factor`, so raw coordinates (∼8 mm range) differ dramatically from pipeline coordinates (∼87 mm range). Running the optimizer on raw coordinates would produce a CTR placement that is geometrically incompatible with the interpolated network. This was a critical bug that was discovered and fixed by moving the optimizer into the pipeline itself (Step 3b), after interpolation.

5 Modified File: Pipeline Orchestrator

5.1 Overview

- **File:** `xcavate/pipeline.py`
- **Status:** [MODIFIED]
- **Changes:** ∼45 lines added/modified across 4 locations

5.2 Change 1: Safe CTR Config Scoping (Line 156)

What: Added `ctr_config = None` initialization before the CTR pipeline branch.

Why: Without this, accessing `ctr_config` in the return dict would raise `NameError` when `printer_type != CTR`. The initialization ensures the variable is always defined regardless of which pipeline branch executes.

5.3 Change 2: Auto-Placement Integration (Step 3b)

What: Added a new pipeline step between interpolation (Step 3) and graph construction (Step 4):

```python
if config.ctr_auto_place:
    from xcavate.core.ctr_placement import optimize_ctr_placement
    opt = optimize_ctr_placement(
        points_xyz,
        radius=config.ctr_radius,
        ss_lim=config.ctr_ss_max,
        ntnl_lim=config.ctr_ntnl_max,
        calibration_file=config.ctr_calibration_file,
    )
    config.ctr_position_cartesian = tuple(opt["position"].tolist())
    config.ctr_position_cylindrical = None
    config.ctr_orientation = tuple(opt["orientation"].tolist())
```

Why: The auto-placement must operate on `points_xyz` *after* interpolation and coordinate scaling. Running it on raw input coordinates produced a placement optimized for an 8 mm-scale network, but the pipeline operates on an 87 mm-scale network. This resulted in 2.8% reachability instead of 100%. Moving the optimizer to Step 3b (after preprocessing, before graph construction) ensures correct coordinate space alignment.

Where: Inserted between the existing interpolation step and the graph construction step in `run_xcavate()`.

5.4 Change 3: Empty Pass Guard in `_compute_instructions()`

What: Changed `first_key = min(print_passes_sm.keys())` with an early-return guard:

```python
if not print_passes_sm:
    return {
        "print_passes_sm": {},
        "print_passes_mm": {},
        ... # stub result dict
    }
first_key = min(print_passes_sm.keys())
```

Why: When CTR reachability filtering removes *all* nodes (e.g., before auto-placement was integrated correctly), `print_passes_sm` becomes empty. Calling `min()` on an empty dict raises

`ValueError`. The guard returns a valid but empty result dict. Additionally, the original code used `print_passes_sm[0][0]` which assumed pass indices start at 0 — this was changed to use `min()` for robustness.

5.5 Change 4: Simulation Data in Return Dict

What: Added two new keys to the pipeline return dictionary:

```
return {
    ...  # existing keys
    "gimbal_solutions": gimbal_solutions,
    "ctr_config_params": {
        "X": ctr_config.X.tolist(),
        "R_hat": ctr_config.R_hat.tolist(),
        "n_hat": ctr_config.n_hat.tolist(),
        "theta_match": float(ctr_config.theta_match),
        "radius": ctr_config.radius,
        "ss_lim": ctr_config.ss_lim,
        "ntnl_lim": ctr_config.ntnl_lim,
    } if ctr_config is not None else None,
}
```

Why: The GUI simulation viewer needs both the per-node gimbal solutions (actuator values) and the CTR geometric parameters (base position, insertion direction, bend radius) to reconstruct the needle geometry for each frame. The full `CTRConfig` object contains non-serializable `scipy.interpolate.interp1d` objects and cannot be stored directly in session state. Instead, we extract only the geometric parameters needed for visualization.

Data structures:

- `gimbal_solutions`: `Dict[int, GimbalNodeSolution]` — maps node index to solved actuator values. Each solution contains `config_idx`, `alpha`, `phi`, `z_ss`, `z_ntnl`, `theta`. Will be `None` if gimbal solver was not used.
- `ctr_config_params`: Serializable dict of CTR geometric parameters. Will be `None` if printer type is not CTR.

6 Modified File: Streamlit GUI Application

6.1 Overview

- **File:** `xcavate/gui/app.py`
- **Status:** [MODIFIED]
- **Changes:** ∼120 lines added/modified across 5 locations

6.2 Change 1: Gimbal Toggle (Top-Level)

What: Added a gimbal toggle immediately below the multimaterial toggle in the sidebar:

```
multimaterial = st.toggle(
    "Multimaterial", value=False, ...)

gimbal_enabled = st.toggle(
    "Gimbal", value=True,
    help="Enable gimbal-based workspace expansion for CTR "
         "printing. Tilts the insertion direction within a cone "
         "to reach fringe nodes. Only applies when Printer "
```

```
            "type = CTR.",
)
```

Why: The gimbal toggle was previously buried inside the CTR settings expander, behind a "Customize CTR settings" checkbox. Since the gimbal is a high-impact feature toggle (it determines whether 435+ fringe nodes are reachable or pruned), it belongs at the top level alongside other major feature flags like multimaterial. The default is `True` because the gimbal expands the workspace at minimal computational cost.

Where: Moved from line 545 (inside CTR customize block) to line 143 (sidebar top-level, immediately after multimaterial toggle).

6.3 Change 2: CTR Default Configuration

What: All CTR parameters are set as plain Python variables with sensible defaults, displayed to the user only when a "Customize CTR settings" checkbox is checked:

```
# Defaults loaded automatically
ctr_radius = 55.0
ctr_ss_max = 65.0
ctr_ntnl_max = 140.0
ctr_ss_axis_name = "X"
ctr_ntnl_axis_name = "Y"
ctr_rot_axis_name = "Z"
ctr_extruder_axis_name = "E0"
ctr_extruder_mm_per_mm = 0.02
ctr_extrusion_feedrate = 600.0
ctr_jogging_feedrate = 1200.0
ctr_needle_body_samples = 20
gimbal_cone_angle = 15.0
gimbal_n_tilt = 3
gimbal_n_azimuth = 8
```

Why: Previously, every CTR parameter required manual entry. Since the hardware specs are fixed (SS 4 mm OD, nitinol 1.6 mm OD, radius 55 mm), the defaults match the physical hardware. Users only need to open the customize panel for non-standard configurations.

6.4 Change 3: Auto-Optimize CTR Placement

What: Added an "Auto-optimize CTR placement" checkbox (default `True`) inside the CTR expander. When enabled, sets `config.ctr_auto_place = True` which triggers the optimizer in the pipeline.

Session state integration:

- `ctr_auto_result`: stores the optimizer's return dict for display
- `ctr_auto_position`: optimized position array
- `ctr_auto_orientation`: optimized direction array

When auto-placement results are available, the GUI displays a success message:

> *"Auto-placement: 18681/18681 nodes reachable (100.0%) from 336 candidates"*

6.5 Change 4: CTR Needle Simulation Section

What: Added a complete simulation viewer section after the 3D Visualization output:

```
sim_mode = st.radio(
    "Simulation mode",
    ["All passes", "Single pass"],
    index=0, horizontal=True,
)
```

Components:

1. **Mode selector:** Radio button choosing "All passes" or "Single pass".
2. **Pass selector:** Dropdown (single-pass mode only) showing pass index and node count.
3. **Animation controls:** Three sliders:
   - Max frames: 50–500 (default 300 for all-passes, 200 for single)
   - Frame speed: 20–200 ms (default 50 ms = 20 fps)
   - Trail detail: 1,000–50,000 points (default 20,000)
4. **Generate button:** Triggers animation building with spinner.
5. **Plotly viewer:** Full-width interactive 3D animation.
6. **Download button:** Exports standalone HTML file.

Fallback logic: When `gimbal_solutions` is `None` (non-gimbal CTR mode), the simulation section calls `build_fallback_solutions()` to generate synthetic actuator values using analytical inverse kinematics. This ensures the simulation works regardless of whether the gimbal solver was enabled.

6.6 Change 5: Config Building

What: Added all CTR and gimbal parameters to `_build_config()`:

```
return XcavateConfig(
    ...
    ctr_auto_place=auto_optimize_ctr,
    ctr_calibration_file=ctr_cal_path,
    ctr_position_cartesian=ctr_position_cartesian,
    ctr_orientation=(ctr_ori_x, ctr_ori_y, ctr_ori_z),
    ctr_radius=ctr_radius,
    ctr_ss_max=ctr_ss_max,
    ctr_ntnl_max=ctr_ntnl_max,
    gimbal_enabled=gimbal_enabled,
    gimbal_cone_angle=gimbal_cone_angle,
    gimbal_n_tilt=gimbal_n_tilt,
    gimbal_n_azimuth=gimbal_n_azimuth,
    ...
)
```

7 Modified File: Configuration Dataclass

7.1 Overview

- **File:** `xcavate/config.py`
- **Status:** [MODIFIED]
- **Changes:** Updated default values for CTR hardware parameters

7.2 Default Value Updates

Table 4: Configuration Default Changes

Parameter	Previous	Current	Reason
`ctr_radius`	47.0	55.0	Match physical hardware
`ctr_ss_od`	(not set)	4.0	SS tube outer diameter
`ctr_ntnl_od`	(not set)	1.6	Nitinol tube outer diameter
`ctr_auto_place`	(not set)	False	New auto-placement flag

The `ctr_auto_place` flag defaults to `False` in the config dataclass (safe default for CLI/API use), but the GUI sets it to `True` via the checkbox.

8 Modified File: Plotting Module

8.1 Overview

- **File:** `xcavate/viz/plotting.py`
- **Status:** [MODIFIED]
- **Changes:** ∼8 lines added (empty-pass guards)

8.2 Empty Pass Guard in `create_network_plot_merged()`

What: Added two guards:

1. Skip empty passes: `if not node_indices: continue`
2. Return empty figure if no segments: guard against `np.vstack([])` crash.

Why: When CTR reachability filtering removes all nodes from certain passes (before gimbal was enabled), the merged plot function received empty pass lists. Calling `np.vstack()` on an empty list raises `ValueError: need at least one array to concatenate`. The guards ensure graceful degradation.

9 Bugs Discovered and Fixed

During implementation, several bugs were discovered and resolved:

9.1 Coordinate Scaling Mismatch (Critical)

Symptom: Auto-placement optimizer reported 100% reachability, but the pipeline only achieved 2.8% (most nodes pruned as unreachable).

Root cause: The optimizer was running on raw input coordinates (∼8 mm range) in the GUI before the pipeline. However, the pipeline's preprocessing step applies `convert_factor` (10.0, cm→mm) × `scale_factor`, producing interpolated coordinates at ∼87 mm range. The CTR placement optimized for the raw scale was geometrically incompatible with the interpolated scale.

Fix: Moved the optimizer into the pipeline itself (Step 3b), running on `points_xyz` after interpolation and coordinate scaling. This ensures the optimizer and the rest of the pipeline operate in the same coordinate space.

9.2 KeyError on Pass Index (Medium)

Symptom: `KeyError: 0` in `_compute_instructions()` at `print_passes_sm[0][0]`.

Root cause: After pass reordering, the pass dictionary keys may not start at 0.

Fix: Changed to `first_key = min(print_passes_sm.keys())`.

9.3 Empty Sequence on Min (Medium)

Symptom: `ValueError: min() arg is an empty sequence` on `min(print_passes_sm.keys())`.

Root cause: All nodes unreachable ⇒ `print_passes_sm` is completely empty.

Fix: Added early return with stub result dict when `not print_passes_sm`.

9.4 Empty Array Concatenation (Low)

Symptom: `ValueError: need at least one array to concatenate` in `np.vstack(segments)` within `create_network_plot_merged()`.

Root cause: Empty passes after CTR filtering.

Fix: Skip empty passes and return empty figure if no segments.

10 Performance Considerations

10.1 HTML File Size Optimization

Table 5: Simulation HTML Size Comparison

Approach	Size	Notes
Naive (all traces per frame)	101 MB	10K ghost points × 300 frames
Optimized (`traces=[2..7]`)	∼36 MB	Static traces excluded from frames

The key optimization is using `traces=[2,3,4,5,6,7]` in each `go.Frame` to only update the 6 dynamic traces, skipping the static ghost network (trace 0, up to 10K points) and base marker (trace 1). This reduced the Plotly JSON payload by approximately 65%.

10.2 Trail Subsampling

The printed trail grows linearly with frame index. To prevent the HTML from ballooning as the trail accumulates:

- Trail is capped at `trail_max_points` (default 20,000 for both modes).
- When the trail exceeds this limit, it is uniformly subsampled while preserving the current endpoint.
- The trail uses `mode="lines+markers"` with 2 px markers so that individual printed positions are visible even when the line is sparse.

10.3 Auto-Placement Optimizer

- 336 candidates evaluated with vectorized `batch_is_reachable()` calls.
- Runtime: < 2 seconds for 18,681-node networks.
- Memory: Each candidate requires a temporary `CTRConfig` allocation (lightweight).

11 Gimbal Workspace Analysis

11.1 Problem: Fringe Node Pruning

Without the gimbal, the CTR workspace is determined by a single insertion direction $\hat{R}$. Nodes near the boundary of the reachable volume are pruned during pathfinding. For the `finale.txt` test network (18,681 nodes), **435 nodes** were unreachable with the base orientation alone.

11.2 Solution: Gimbal Cone Tilt

The gimbal enables the CTR to tilt $\hat{R}$ by angle $\alpha \in [0, \alpha_{\max}]$ at azimuth $\phi \in [0, 2\pi)$. With the default configuration:

- Cone half-angle: 15°
- Tilt levels: 3 (at 5°, 10°, 15°)
- Azimuth samples: 8 (at 0°, 45°, 90°, ...)
- Total gimbal configurations: $1 + 3 \times 8 = 25$ (including the untilted base)

Each tilted configuration shifts the reachable workspace boundary. A fringe node that exceeds the radial calibration domain for the base $\hat{R}$ may fall within the domain of a tilted $\hat{R}'$. The gimbal solver tries each configuration per node and selects the first that provides a valid IK solution.

11.3 Result

With gimbal enabled and auto-placement:

- **Reachable nodes:** 18,681 / 18,681 (**100%**)
- **Nodes pruned:** 0 (previously 435)
- **Print passes:** 101
- **Gimbal solutions:** 18,681 (one per node)
- **Warnings:** 0

12 Data Flow Architecture

The complete data flow for CTR simulation is:

1. **GUI** (`app.py`): User uploads network file, sets printer type to CTR, enables gimbal, clicks Run.
2. **Config** (`config.py`): `_build_config()` assembles `XcavateConfig` with all CTR and gimbal parameters.
3. **Pipeline** (`pipeline.py`):
   (a) Read and interpolate network → `points_xyz`
   (b) **Step 3b:** Auto-place CTR → update `config.ctr_position_cartesian` and `config.ctr_orientat`
   (c) Build graph, find passes with gimbal solver → `print_passes_sm`, `gimbal_solutions`
   (d) Store `ctr_config_params` in return dict
4. **GUI** (`app.py`): Receives pipeline result. User clicks "Generate Simulation".
5. **Simulation** (`ctr_simulation.py`):
   (a) Reconstruct `CTRConfig` from stored config
   (b) For each animation frame: look up `gimbal_solutions[node]` → compute gimbal frame → compute needle geometry

(c) Build `go.Figure` with 8 traces and N frames

6. **GUI** (`app.py`): Display interactive Plotly animation. Offer HTML download.

13 Key Data Structures

13.1 GimbalNodeSolution

```
@dataclass
class GimbalNodeSolution:
    config_idx: int    # index into gimbal configs list
    alpha: float       # tilt angle (radians)
    phi: float         # azimuth angle (radians)
    z_ss: float        # stainless steel extension (mm)
    z_ntnl: float      # nitinol extension (mm)
    theta: float       # rotation angle (radians)
```

One solution per printed node. Used by both the G-code writer (to emit actuator commands) and the simulation engine (to reconstruct needle geometry).

13.2 ctr_config_params

```
{
    "X": [x, y, z],             # (3,) base position
    "R_hat": [rx, ry, rz],      # (3,) insertion direction
    "n_hat": [nx, ny, nz],      # (3,) bend normal
    "theta_match": float,        # alignment angle
    "radius": float,             # bend radius (mm)
    "ss_lim": float,             # max SS extension (mm)
    "ntnl_lim": float,           # max nitinol extension (mm)
}
```

Serializable representation of CTR geometric parameters for session state storage.

14 Testing and Validation

14.1 Pipeline Test: `finale.txt`

The full pipeline was run on the `finale.txt` test network (18,681 unique nodes after interpolation):

Table 6: Pipeline Test Results

Metric	Value
Total unique nodes	18,681
Nodes printed	18,882 (includes overlaps from gap closure)
Nodes pruned (unreachable)	0
Print passes	101
Gimbal solutions generated	18,681
Auto-placement candidates tested	336
Reachable fraction	100.0%
Pipeline warnings	0

14.2 Without Gimbal (Baseline)

When gimbal is disabled, 435 fringe nodes are pruned from the workspace boundary. These nodes have perpendicular distances from the insertion axis that exceed the radial calibration domain for the single base orientation.

14.3 Simulation Verification

The simulation was verified by:

1. Confirming the SS tube (orange) extends and retracts correctly as nodes change.
2. Confirming the nitinol arc (blue) curves in the correct bend plane.
3. Confirming the tip marker (green) traces the actual print path.
4. Confirming gimbal direction lines (cyan vs. gray) diverge at tilted configurations.
5. Confirming per-pass coloring in all-passes mode correctly segments the trail.
6. Verifying standalone HTML export works in browser without Streamlit.

15 Complete File Inventory

Table 7: All Files Created or Modified

File Path	Status	Lines	Role
`xcavate/viz/ctr_simulation.py`	r	861	Animation engine
`xcavate/core/ctr_placement.py`	r	174	Placement optimizer
`xcavate/pipeline.py`	**[MODIFIED]**	+45	Pipeline orchestration
`xcavate/gui/app.py`	**[MODIFIED]**	+120	GUI integration
`xcavate/config.py`	**[MODIFIED]**	+5	Config defaults
`xcavate/viz/plotting.py`	**[MODIFIED]**	+8	Empty-pass guard

Supporting files (not modified in this change set, but referenced):

- `xcavate/core/ctr_kinematics.py` — CTR forward/inverse kinematics, `CTRConfig` class
- `xcavate/core/gimbal_solver.py` — `GimbalNodeSolution` dataclass, gimbal pathfinding
- `xcavate/core/ctr_collision.py` — Collision detection for CTR needle body
- `xcavate/core/oracle.py` — Diagnostic detector, gimbal config builder
- `xcavate/io/gcode/ctr.py` — CTR G-code writer with gimbal transitions

16 Conclusion

The changes documented in this report transform the X-CAVATE CTR pipeline from a compute-only system into an interactive visualization and optimization platform. The key engineering contributions are:

1. A self-contained forward kinematics module (`ctr_simulation.py`) that computes needle geometry from actuator values without depending on the full kinematics pipeline or scipy interpolation objects.
2. An automated placement optimizer (`ctr_placement.py`) that eliminates manual trial-and-error positioning of the CTR base, achieving 100% node reachability on test networks.

3. Gimbal integration at the GUI level with a top-level toggle and sensible defaults, making workspace expansion accessible without requiring deep knowledge of the CTR configuration space.
4. Robust pipeline integration with proper coordinate-space alignment, empty-pass handling, and serializable data structures for session state persistence.
5. Performance-optimized Plotly animations using selective trace updates, trail subsampling, and configurable frame limits to keep HTML files manageable ($\sim$36 MB for 300-frame all-passes simulations on 18K-node networks).